Working on the Cluster : The Basics

Logging into the Cluster

You can either `ssh` into the cluster through VSCode or directly through your powershell/terminal. See the VS Code Remote–SSH documentation for instructions on SSH windows for VSCode.

Note: After connecting, you are on the login node. **Do not run anything on the login node**. The login node is only for basic file/software management and connecting to worker nodes. The worker nodes are where you will run your programs. Choking up the login node is a great way to get kicked off the cluster. The SDSU cluster uses SLURM to schedule jobs on the worker nodes. We'll describe the basics of using SLURM in a later section.

File Transfers

You will frequently need to move files (code scripts, data) between the cluster and your local machine. This can either be done through the command line (`scp`) or through a GUI like WinSCP.

The general format for `scp` from your powershell/terminal is

```
scp <path/to/localfile> username@hostname:<path/to/destinationfile>
```

The above moves the file from your local machine to the cluster. If you'd like to move the contents of entire folder,

```
scp <path/to/localfolder/*> username@hostname:<path/to/destinationfolder/>
```

To move files in reverse (download from the cluster to your local machine), simply reverse the ordering

```
scp username@hostname:<path/to/downloadfile> <path/to/localfile>

scp username@hostname:<path/to/downloadfolder/*> <path/to/localfolder/>
```

SLURM basics

Many users share cluster resources, so there needs to be some protocol to ration resources to users. This is the job of the *scheduler*. The SDSU cluster uses the SLURM program as the scheduler. The basic process is

- You request a worker with a set amount of resources (# of nodes, # of cores).
- If those resources are available, your job runs (or interactive session starts) immediately. Otherwise, you are "in line", and your job will start when it is your turn.

There are two types of SLURM submissions: batch jobs and interactive sessions.

Batch Jobs

In a batch job, you submit a script or executable to be ran on a worker with a specified amount of resources. The script will run as soon as the resources are available and dump the output in a `.out` file. This is the sensible option when you are pretty sure your script works and/or you are requesting a lot of resources (so

you will probably need to wait in line). However, if you are still developing/benchmarking/profiling, this might be an irritating way to work. In this case, you probably want an interactive session (next section).

Batch jobs are submitted by running `.sbatch` scripts. For any example, let's say you want to run the Julia script `my_test.jl` with 1 CPU node using 8 cores. Your batch script, call it `my_test.sbatch` may look like this

```
#!/bin/bash
#SBATCH --job-name=my_testy_test # The name of your job.
#SBATCH --output=test_output.out # Desired name of your output dump.
#SBATCH --time=00:10:00 # Your requested max wall-time.
#SBATCH --nodes=1 # Number of physical machines.
#SBATCH --ntasks=1 # Number of independent tasks.
#SBATCH --cpus-per-task=8 # Number of cores
#SBATCH --partition=compute # "compute" -> CPU nodes

module load julia # Load Julia into your env.

julia my_test.jl
```

You can find the full SLURM cheatsheet (PDF), but you won't need anything more sophisticated than the above. Mostly we'll just be playing around with the number of cores, `--cpus-per-task`, and maybe using the GPU nodes later in the course, `--partition=gpu`.

You can then run the batch script from your terminal like

```
sbatch my_test.sbatch
```

and wait for the output `.out` file to drop. If you want to see if your job is running (or where you are in line), you can check the queue with

```
squeue -u <your-user-name-here>
```

If your job name is listed with an `R` under `ST` (short for status), it is running. If there is a `PD` under `ST`, then your job is pending, awaiting your requested resources.

Interactive Sessions

You can start an interactive session on a worker node with the `srun` command. The general format is

```
srun <requested-resources-here> --pty bash
```

For example, let's start a session on a single compute node with 8 cores.

```
srun --nodes=1 --ntasks=1 --cpus-per-task=8 --partition=compute --pty bash
```

Once those resources are available, your terminal prompt should be something like

```
[pmay@SDSMT.LOCAL@node017 ~]$
```

but with your username instead of mine, and whatever node you were allocated instead of `node017`. Command `srun` has short options, if you'd like to save some typing. For example, the below is an equivalent command

```
srun -N 1 -n 1 -c 8 -p compute --pty bash
```

In this class, your next step will pretty much always be to load julia with `module load julia`. Then you can run your Julia scripts from the terminal with the `julia` command.